Spectral-Domain Bayesian Audio Signal Processing

Reilly

March 2026

Abstract

This report documents the development of a spectral-domain Bayesian approach to audio signal denoising and source separation. The project implements Wiener filtering with learned priors in the Short-Time Fourier Transform (STFT) domain. We begin with fundamental signal generation and spectral analysis, building toward a full Bayesian denoising pipeline.

Contents

1 Introduction **2**

2 Signal Generation and Spectral Analysis **2**

2.1 Composite Signal Generation 2

2.2 The Discrete Fourier Transform 3

2.3 Frequency Spectrum Visualisation 4

3 Next Steps **4**

1 Introduction

The goal of this project is to implement a Bayesian framework for audio signal processing that operates in the spectral domain. The core technique is Wiener filtering, where the optimal filter gain at each time-frequency bin is determined by estimates of the signal and noise power spectral densities (PSDs). By placing priors on these PSDs, we obtain a principled, probabilistic approach to denoising and source separation.

2 Signal Generation and Spectral Analysis

2.1 Composite Signal Generation

We begin by constructing a test signal composed of multiple sinusoidal components. The signal is defined as a sum of sinusoids:

$$x(t) = \sum_i A_i \sin(2\pi f_i t + \phi_i) \tag{1}$$

where A_i, f_i, and ϕ_i are the amplitude, frequency, and random phase offset of each component. The random phase ensures a more realistic signal that does not start with all components aligned at zero. Our test signal uses three components:

Component	Amplitude	Frequency (Hz)	Phase
1	0.67	320	$\phi_1 \sim \mathcal{U}(0, 2\pi)$
2	0.74	240	$\phi_2 \sim \mathcal{U}(0, 2\pi)$
3	1.00	160	$\phi_3 \sim \mathcal{U}(0, 2\pi)$

The signal is sampled at $f_s = 15000$ Hz over a duration of 0.1 seconds, producing $N = 1500$ samples.

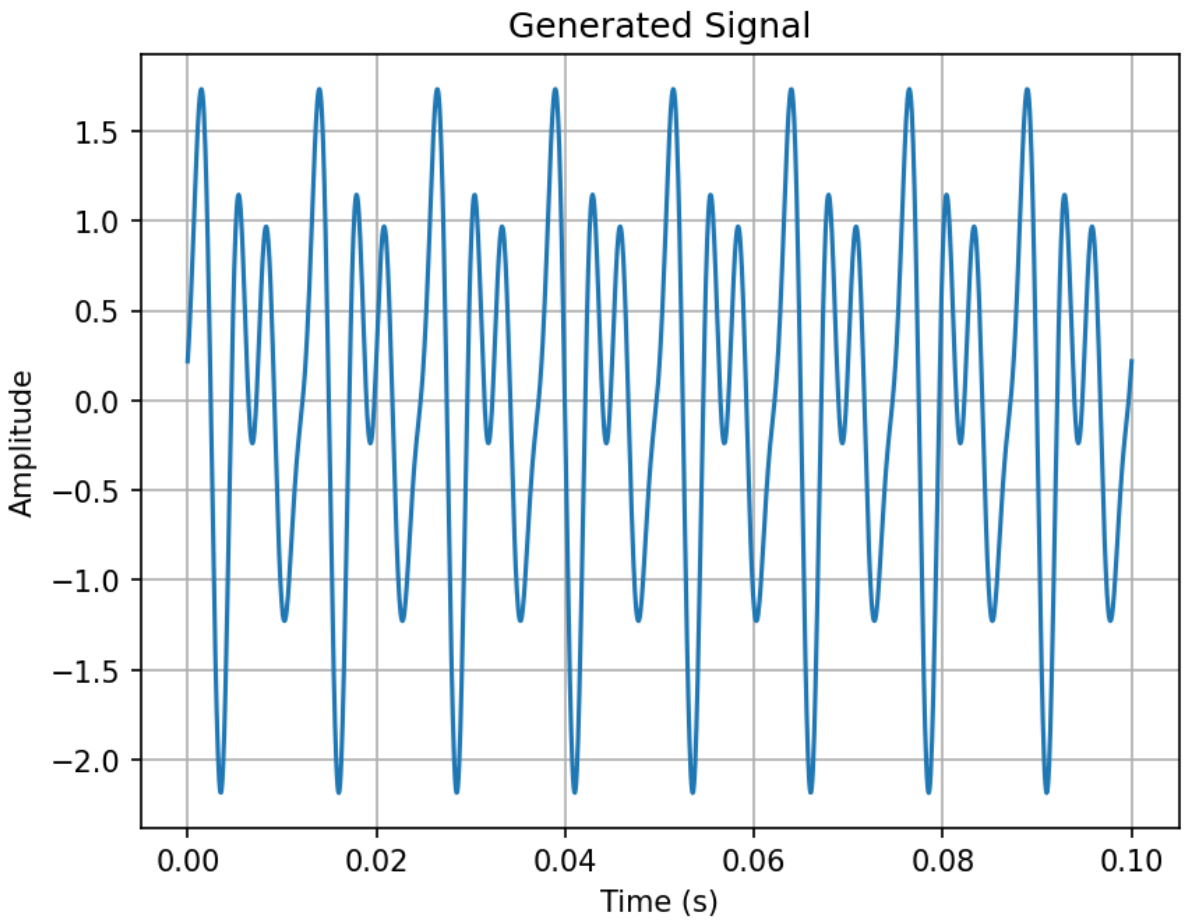

Figure 1: Time-domain representation of the composite test signal.

2.2 The Discrete Fourier Transform

To analyse the frequency content of the signal, we implement the Discrete Fourier Transform (DFT) directly from its definition:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{2}$$

Rather than using nested loops, the implementation vectorises this computation using NumPy broadcasting. The indices k and n are formed as a column vector and row vector respectively, producing the full $N \times N$ twiddle factor matrix in a single operation:

```python
def DFT(x):
    N = len(x)
    n = np.arange(N)
    k = np.arange(N).reshape(-1, 1)
    X = np.sum(x * np.exp(-2j * np.pi * k * n / N), axis=1)
    return X
```

Listing 1: Vectorised DFT implementation

2.3 Frequency Spectrum Visualisation

The magnitude spectrum is computed as $|X[k]|/N$ and displayed as a bar chart over the single-sided frequency range $[0, f_s/2)$. The frequency axis is automatically scaled to focus on bins with significant energy, defined as those exceeding 1% of the peak magnitude.

The three frequency components at 160, 240, and 320 Hz are clearly resolved in the resulting spectrum, with magnitudes proportional to their respective amplitudes.

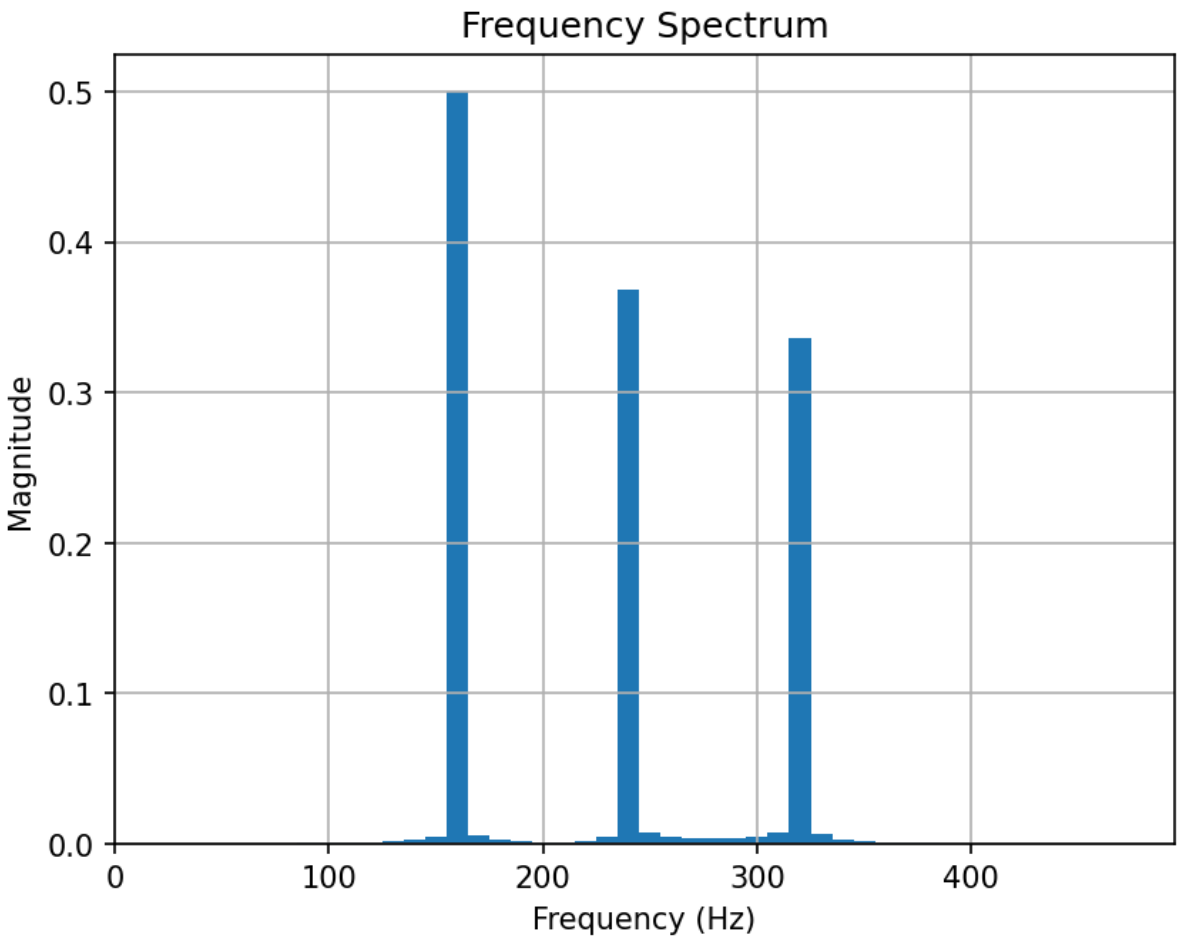

Figure 2: Single-sided magnitude spectrum of the test signal, showing peaks at 1, 2, and 10 Hz.

3 Next Steps

The following stages of the project will build on this spectral analysis foundation:

1. **STFT implementation** — windowed, overlapping spectral analysis for time-varying signals.
2. **Wiener filter** — optimal spectral-domain gain computation given signal and noise PSD estimates.
3. **Noise prior models** — stationary, adaptive, and NMF-based prior estimation.

4. **Variational inference** — full Bayesian treatment with uncertainty quantification.
5. **Source separation** — multi-source Wiener filtering for mixed audio signals.